

Foundations of AI

Lab Experiments.

1. Water Jug:-

Problem Statement:-

* 2 water Jugs are given with capacity of max 3 and 4 liters.

* Initially both are empty & have no markings and possible operations are:

i) Fill x fully

iv) Empty y

ii) Fill y completely

v) Pour x to y till y is full or x is empty

iii) Empty x

vi) Pour y to x till y is empty or x is full.

Each operation represents a state (x, y) where x & y are amount of liters.

→ The objective is to generate all possible successor state from given state, to expand all states using BFS & find no of levels to reach desired output & to determine no of New states at each level.

Solution:-

The Successor (state) fn generates all possible next states by applying 6 operations: fill x , fill y , empty x , empty y , Pour $x \rightarrow y$, Pour $y \rightarrow x$.

* The Program uses BFS to explore state space level by level.

* We use "List" to store states of current level & next level. To avoid getting repeated node visits we use if ns not in seen:

* If the state is new, it is added to seen, and new, this ignores already generated nodes

* A tree will generate 6, 36, 216, 1296 nodes due to Combinatorial explosion. However, by storing only unique values, the program finds only 20 unique states.

Python Code:-

def Successor (state):

x, y = state

pour_x_to_y = min(x, 3-y)

pour_y_to_x = min(y, 4-x)

return [(4, y), (x, 3), (0, y), (0, x), (x-pour_x_to_y + pour_x_to_y, x+pour_y_to_x - pour_y_to_x)]

start = (0, 0)

Seen = {start}

frontier = [start]

level = 0

while frontier and level < 8:

new = []

for s in frontier:

for ns in Successor(s):

if ns not in Seen:

Seen.add(ns)

new.append(ns)

if ns[0] == 2: Print('Goal: 2L in x at level', level+1)

level = level + 1

Print ("Level", level, "->", len(new), "new states:", Sorted(new))

frontier = new

Print ("Total states found:", len(Seen))

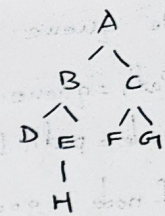
Lab 2a: Breadth-First Search (BFS)

Problem Statement:-

* To find Path from A to H.

* Return Path as a list.

* Display order in which the nodes are visited using BFS.



Solution:-

* The BFS starts from node A & explores graph level by level. A queue is used to store the paths that are yet to be explored. At each step, 1st path is removed from queue & last node is checked.

* If goal (H) is returned Path else all unvisited neighbouring nodes are added to queue as new paths.

* The visited nodes are stored in "visited list" which prevents visiting same nodes.

Py Code:-

graph = { 'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F', 'G'], 'D': [], 'E': ['H'], 'F': [], 'G': [], 'H': [] }

def bfs (start, goal):

queue = [start]

visited = []

while queue:

Path = queue.pop(0)

node = path[-1]

if node == goal:

return path, visited

if node not in visited:

visited.append(node)

for n in graph[node]:

queue.append(path + [n])

return None, visited

P, V = bfs('A', 'H')

Print("BFS path: ", P)

Print("BFS visited: ", V)

Ans:

1) BFS & DFS differ by exactly 1 line:

BFS:

path = queue.pop(0)

DFS:

path = queue.pop()

2) Why these lines change the algorithm?

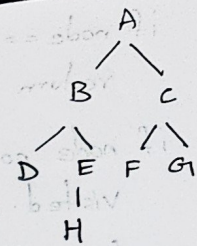
In BFS, the 1st inserted path is expanded first, so search proceeds level by level & finds shortest path in unweighted graph.

In DFS, the most recently inserted path is expanded first, search follows one branch as deep as possible before backtracking.

Lab 2 b). Depth First Search.

Problem Statement:-

- To find path from A to H using DFS.
- Return Path as list
- Display order of visited nodes



Solution:-

- We use Stack to store paths from node A & explores a branch completely before moving to another branch.

branch.

- It used LIFO.

- The visited list stores all visited nodes to avoid visiting same nodes.

- The statement reversed(graph[node]) pushes right child first & left child last, ensuring right left is explored 1st when popped from stack.

Py Code:-

```
graph = {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F', 'G'], 'D': [], 'E': ['H'], 'F': [], 'G': [], 'H': []}
```

def stack_dfs(start, goal):

stack = [[start]]

visited = []

while stack:

path = stack.pop()

node = path[-1]

if node == goal:
return path, visited

if node not in visited:
visited.append(node)

for n in reversed(graph[node]):

Stack.append(path + [n])

return None, visited.

P, v = dfs('A', H)

Print("DFS path :", P)

Print("DFS visited", v).

output:-

DFS path: ['A', 'B', 'E', 'H']

DFS visited: ['A', 'B', 'D', 'E']

Qn: DFS visits fewer node than BFS. Is DFS better?

No, DFS isn't always better because visiting fewer nodes depends on position of goal node.

If goal is a leaf node that is extended infinitely & the DFS spend time only in that branch.

But BFS searches all nodes breadth wise which is mostly better than DFS.

(loop, trail2) = trail2.pop()

stack = [trail2]

visited = set()

while stack:

path = stack.pop()

node = path[-1]